



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

LAB EXPERIMENTS

NAME: R. RACHITHA

REGNO: 192421058

COURSE CODE: ITA0402

COURSE NAME: STATISTICS WITH R PROGRAMMING

8. Generate random numbers from a normal distribution; count occurrences.

```
set.seed(123)

random_numbers <- rnorm(100, mean = 50, sd = 10)

print("Random Numbers")

print(random_numbers)

breaks <- seq(20, 80, by = 10)

count <- table(cut(random_numbers, breaks))

print("Frequency Count")

print(count)

summary(random_numbers)
```

```
Console Terminal Background Jobs
R - R 4.6.1 - C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB/Exp 8/
[1] "Random Numbers"
> print(random_numbers)
[1] 44.39524 47.69823 65.58708 50.70508 51.29288 67.15065 54.60916 37.34939 43.13147 45.54338
[11] 62.24082 53.59814 54.00771 51.10683 44.44159 67.86913 54.97850 30.33383 57.01356 45.27209
[21] 39.32176 47.82025 39.73996 42.71109 43.74961 33.13307 58.37787 51.53373 38.61863 62.53815
[31] 54.26464 47.04929 58.95126 58.78133 58.21581 56.88640 55.53918 49.38088 46.94037 46.19529
[41] 43.05293 47.92083 37.34604 71.68956 62.07962 38.76891 45.97115 45.33345 57.79965 49.16631
[51] 52.53319 49.71453 49.57130 63.68602 47.74229 65.16471 34.51247 55.84614 51.23854 52.15942
[61] 53.79639 44.97677 46.66793 39.81425 39.28209 53.03529 54.48210 50.53004 59.22267 70.50085
[71] 45.08969 26.90831 60.05739 42.90799 43.11991 60.25571 47.15227 37.79282 51.81303 48.61109
[81] 50.05764 53.85280 46.29340 56.44377 47.79513 53.31782 60.96839 54.35181 46.74068 61.48808
[91] 59.93504 55.48397 52.38732 43.72094 63.60652 43.99740 71.87333 65.32611 47.64300 39.73579
> breaks <- seq(20, 80, by = 10)
> count <- table(cut(random_numbers, breaks))
> print("Frequency Count")
[1] "Frequency Count"
> print(count)
(20,30] (30,40] (40,50] (50,60] (60,70] (70,80]
1 13 34 35 14 3
> summary(random_numbers)
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 26.91   45.06   50.62   50.90   56.92   71.87
```

9. Create empty plots with specified axis limits.

```
plot(NULL,
      xlim = c(0, 100),
      ylim = c(0, 100),
      xlab = "X-Axis",
      ylab = "Y-Axis",
      main = "Empty Plot")

points(20, 40, col = "red", pch = 19)

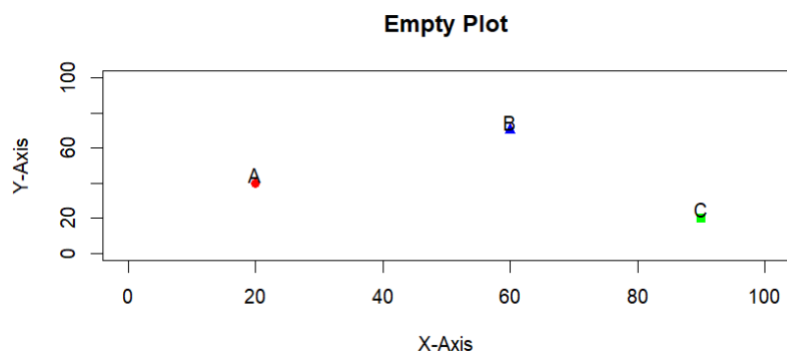
points(60, 70, col = "blue", pch = 17)

points(90, 20, col = "green", pch = 15)

text(20, 45, "A")

text(60, 75, "B")

text(90, 25, "C")
```



10. Create and explore a data frame exam_data with name, score, attempts, and qualify fields. Perform extract, add row/column, sort, save to file.

```
exam_data <- data.frame(
  Name = c("Arun","Bala","Chitra","Divya","Eswar"),
  Score = c(78,92,85,67,90),
  Attempts = c(1,2,1,3,2),
  Qualify = c("Yes","Yes","Yes","No","Yes")
)
print("Original Data Frame")
print(exam_data)
print("Scores")
print(exam_data$Score)
print("Second Row")
print(exam_data[2,])
new_row <- data.frame(
  Name="Farhan",
  Score=88,
  Attempts=1,
  Qualify="Yes"
)
exam_data <- rbind(exam_data,new_row)
exam_data$Grade <- c("B","A","A","C","A","B")
print("Updated Data Frame")
print(exam_data)
sorted_data <- exam_data[order(-exam_data$Score),]
print("Sorted Data")
print(sorted_data)

write.csv(sorted_data,
  "exam_data.csv",
  row.names=FALSE)

print("CSV File Saved Successfully")
```

```
Console Terminal Background Jobs
R • R 4.6.1 • C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB
[1] "Updated Data Frame"
> print(exam_data)
  Name Score Attempts Qualify Grade
1  Arun   78         1     Yes    B
2  Bala   92         2     Yes    A
3 Chitra   85         1     Yes    A
4  Divya   67         3      No    C
5  Eswar   90         2     Yes    A
6 Farhan   88         1     Yes    B
> sorted_data <- exam_data[order(-exam_data$Score),]
> print("Sorted Data")
[1] "Sorted Data"
> print(sorted_data)
  Name Score Attempts Qualify Grade
2  Bala   92         2     Yes    A
5  Eswar   90         2     Yes    A
6 Farhan   88         1     Yes    B
3 Chitra   85         1     Yes    A
1  Arun   78         1     Yes    B
4  Divya   67         3      No    C
>
> write.csv(sorted_data,
+           "exam_data.csv",
+           row.names=FALSE)
>
> print("CSV File Saved Successfully")
[1] "CSV File Saved Successfully"
```

11. Write an R program to read a .csv file and display contents.

```
data <- read.csv("exam_data.csv")

print("Contents of CSV File")

print(data)

str(data)

summary(data)
```

```
Console Terminal Background Jobs
R • R 4.6.1 • C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB/Exp 8/
[1] "Contents of CSV File"
>
> print(data)
  Name Score Attempts Qualify Grade
1  Bala   92         2     Yes    A
2  Eswar   90         2     Yes    A
3 Farhan   88         1     Yes    B
4 Chitra   85         1     Yes    A
5  Arun   78         1     Yes    B
6  Divya   67         3      No    C
>
> str(data)
'data.frame':   6 obs. of  5 variables:
 $ Name   : chr  "Bala" "Eswar" "Farhan" "Chitra" ...
 $ Score  : int  92 90 88 85 78 67
 $ Attempts: int   2  2  1  1  1  3
 $ Qualify : chr   "Yes" "Yes" "Yes" "Yes" ...
 $ Grade  : chr   "A" "A" "B" "A" ...
>
> summary(data)
      Name      Score      Attempts      Qualify      Grade
Length:6      Min.   :67.00      Min.   :1.000      Length:6      Length:6
N.unique:6      1st Qu.:79.75      1st Qu.:1.000      N.unique:2      N.unique:3
N.blank:0      Median:86.50      Median:1.500      N.blank:0      N.blank:0
Min.nchar:4     Mean:83.33      Mean:1.667      Min.nchar:2     Min.nchar:1
Max.nchar:6     3rd Qu.:89.50      3rd Qu.:2.000      Max.nchar:3     Max.nchar:1
      Max.   :92.00      Max.   :3.000
```

12. Perform data reshaping on airquality dataset: melt, cast, compute monthly averages for Ozone, Solar.R, Wind, and Temperature.

```
data("airquality")

print("First 6 Rows of airquality Dataset")
head(airquality)
print("Structure of Dataset")
str(airquality)
print("Summary")
summary(airquality)

monthly_average <- aggregate(
  cbind(Ozone, Solar.R, Wind, Temp) ~ Month,
  data = airquality,
  FUN = mean,
  na.rm = TRUE
)
print("Monthly Average")
print(monthly_average)

print("Average Ozone by Month")
aggregate(Ozone ~ Month,
  data = airquality,
  mean,
  na.rm = TRUE)
print("Average Solar Radiation by Month")
aggregate(Solar.R ~ Month,
  data = airquality,
  mean,
  na.rm = TRUE)
print("Average Wind by Month")
aggregate(Wind ~ Month,
  data = airquality,
  mean,
  na.rm = TRUE)
print("Average Temperature by Month")
aggregate(Temp ~ Month,
  data = airquality,
  mean,
  na.rm = TRUE)
```

```
Console Terminal Background Jobs
R • R 4.6.1 • C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R
[1] "Average Solar Radiation by Month"
> aggregate(Solar.R ~ Month,
+           data = airquality,
+           mean,
+           na.rm = TRUE)
      Month      Solar.R
1      5 181.2963
2      6 190.1667
3      7 216.4839
4      8 171.8571
5      9 167.4333
> print("Average Wind by Month")
[1] "Average Wind by Month"
> aggregate(Wind ~ Month,
+           data = airquality,
+           mean,
+           na.rm = TRUE)
      Month      Wind
1      5 11.622581
2      6 10.266667
3      7  8.941935
4      8  8.793548
5      9 10.180000
> print("Average Temperature by Month")
[1] "Average Temperature by Month"
> aggregate(Temp ~ Month,
+           data = airquality,
+           mean,
+           na.rm = TRUE)
      Month      Temp
1      5 65.54839
2      6 79.10000
3      7 83.90323
4      8 83.96774
5      9 76.90000
```

13. Combine multiple arrays row-wise.

```
array1 <- matrix(1:12, nrow = 3, byrow = TRUE)
array2 <- matrix(13:24, nrow = 3, byrow = TRUE)

print("Array 1")
print(array1)

print("Array 2")
print(array2)

combined_array <- rbind(array1, array2)

print("Combined Array")

print(combined_array)
```

```
Console Terminal Background Jobs
R 4.6.1 C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING
[1] "Array 1"
> print(array1)
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
>
> print("Array 2")
[1] "Array 2"
> print(array2)
     [,1] [,2] [,3] [,4]
[1,]   13   14   15   16
[2,]   17   18   19   20
[3,]   21   22   23   24
>
> combined_array <- rbind(array1, array2)
>
> print("Combined Array")
[1] "Combined Array"
>
> print(combined_array)
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
[4,]   13   14   15   16
[5,]   17   18   19   20
[6,]   21   22   23   24
```

14. Explore and manipulate ChickWeight dataset (sorting, melting, casting by Diet).

```
data(ChickWeight)

sort_data <- ChickWeight[order(ChickWeight$weight), ]
head(sort_data)

library(reshape2)
melt_data <- melt(ChickWeight, id.vars = c("Chick", "Time", "Diet"))
head(melt_data)

cast_data <- dcast(melt_data, Diet ~ variable,
                  value.var = "value", fun.aggregate = mean)
print(cast_data)
```

```
Console Terminal Background Jobs
R 4.6.1 C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB/Exp 13/
> head(sort_data)
  weight Time Chick Diet
196    35    2    18   1
26     39    2     3   1
195    39    0    18   1
293    39    0    27   2
305    39    0    28   2
317    39    0    29   2
>
> library(reshape2)
> melt_data <- melt(ChickWeight, id.vars = c("Chick", "Time", "Diet"))
> head(melt_data)
   Chick Time Diet variable value
1      1    0    1  weight    42
2      1    2    1  weight    51
3      1    4    1  weight    59
4      1    6    1  weight    64
5      1    8    1  weight    76
6      1   10    1  weight    93
>
> cast_data <- dcast(melt_data, Diet ~ variable,
+                   value.var = "value", fun.aggregate = mean)
> print(cast_data)
   Diet weight
1      1 102.6455
2      2 122.6167
3      3 142.9500
4      4 135.2627
```

15. Perform EDA on iris dataset: dimensions, summary, standard deviation, quantiles, grouping by Species, pivot table, categorical grouping with Sepal.Length categories.

```
data(iris)

dim(iris)

summary(iris)

sapply(iris[1:4], sd)

sapply(iris[1:4], quantile)

aggregate(. ~ Species, iris, mean)

table(iris$Species)

iris$Sepal.Category <- cut(iris$Sepal.Length,
                          breaks = c(4, 5, 6, 7, 8),
                          labels = c("Low", "Medium", "High", "Very High"))

table(iris$Sepal.Category, iris$Species)
```

```
Console Terminal Background Jobs
R 4.6.1 C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB/Exp 13/
> dim(iris)
[1] 150 5
>
> summary(iris)
   Sepal.Length   Sepal.Width   Petal.Length   Petal.Width   Species
Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100   setosa   :50
1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300   versicolor:50
Median :5.800   Median :3.000   Median :4.350   Median :1.300   virginica :50
Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
>
> sapply(iris[1:4], sd)
   Sepal.Length   Sepal.Width   Petal.Length   Petal.Width
0.8280661      0.4358663      1.7652982      0.7622377
>
> sapply(iris[1:4], quantile)
   Sepal.Length Sepal.Width Petal.Length Petal.Width
0%           4.3           2.0           1.00          0.1
25%          5.1           2.8           1.60          0.3
50%          5.8           3.0           4.35          1.3
75%          6.4           3.3           5.10          1.8
100%         7.9           4.4           6.90          2.5
>
> aggregate(. ~ Species, iris, mean)
   Species Sepal.Length Sepal.Width Petal.Length Petal.Width
1  setosa      5.006      3.428      1.462      0.246
2 versicolor  5.936      2.770      4.260      1.326
3  virginica  6.588      2.974      5.552      2.026
>
> table(iris$Species)
   setosa versicolor virginica
      50         50         50
```

16. Explore USArrests dataset: summary statistics, state with largest rape arrests, max & min murder rates, correlation among features, states above median assault arrests and bottom 25% for murder, visualization with histogram, density, scatterplots, bar graphs.

```
data(USArrests)

summary(USArrests)

rownames(USArrests)[which.max(USArrests$Rape)]

max(USArrests$Murder)
min(USArrests$Murder)

cor(USArrests)

USArrests[USArrests$Assault > median(USArrests$Assault), ]

USArrests[USArrests$Murder <= quantile(USArrests$Murder, 0.25), ]

hist(USArrests$Murder)

plot(density(USArrests$Murder))

plot(USArrests$Murder, USArrests$Assault)

barplot(USArrests$Murder)
```

```
Console Terminal Background Jobs
R - R 4.6.1 - C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB/Exp 13/ ->

> summary(USArrests)
      Murder      Assault      UrbanPop      Rape
Min.   : 0.800   Min.   : 45.0   Min.   :32.00   Min.   : 7.30
1st Qu.: 4.075   1st Qu.:109.0   1st Qu.:54.50   1st Qu.:15.07
Median : 7.250   Median :159.0   Median :66.00   Median :20.10
Mean   : 7.788   Mean   :170.8   Mean   :65.54   Mean   :21.23
3rd Qu.:11.250   3rd Qu.:249.0   3rd Qu.:77.75   3rd Qu.:26.18
Max.   :17.400   Max.   :337.0   Max.   :91.00   Max.   :46.00

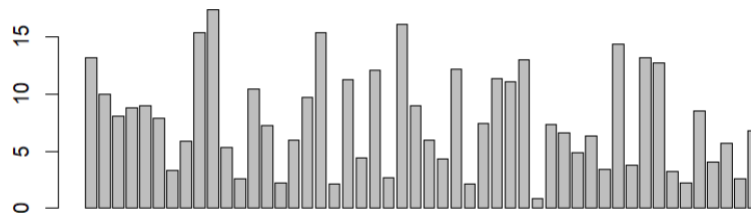
> rownames(USArrests)[which.max(USArrests$Rape)]
[1] "Nevada"

> max(USArrests$Murder)
[1] 17.4

> min(USArrests$Murder)
[1] 0.8

> cor(USArrests)
      Murder      Assault      UrbanPop      Rape
Murder  1.00000000  0.8018733  0.06957262  0.5635788
Assault  0.80187331  1.0000000  0.25887170  0.6652412
UrbanPop 0.06957262  0.2588717  1.00000000  0.4113412
Rape     0.56357883  0.6652412  0.41134124  1.00000000

> USArrests[USArrests$Assault > median(USArrests$Assault), ]
      Murder      Assault      UrbanPop      Rape
Alabama    13.2      236      58 21.2
Alaska     10.0      263      48 44.5
Arizona     8.1      294      80 31.0
Arkansas    8.8      190      50 19.5
California  9.0      276      91 40.6
```



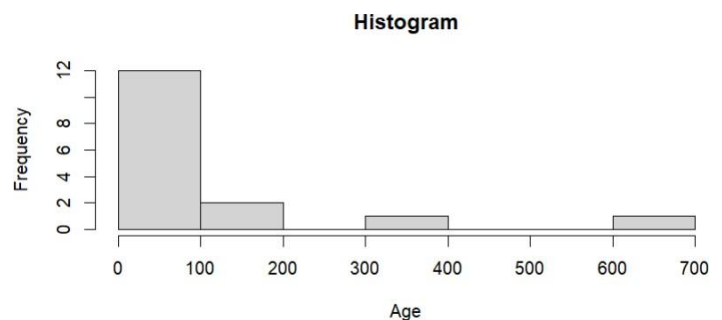
17. Explore Titanic dataset: bar chart of survival vs class, modify plot by gender, histogram of Age.

```
data(Titanic)

barplot(apply(Titanic, c(2, 4), sum),
        beside=TRUE, main="Survival vs Class")

barplot(apply(Titanic, c(1, 4), sum),
        beside=TRUE, main="Survival by Gender")

hist(Titanic[, ,1], main="Histogram", xlab="Age")
```



18. Create graphs in R: boxplot, histogram, bar plot, line chart, scatter plot.

```
data(iris)

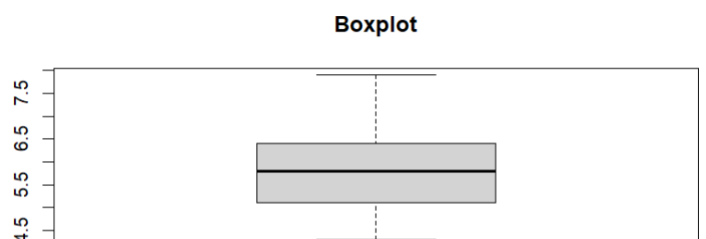
boxplot(iris$Sepal.Length, main="Boxplot")

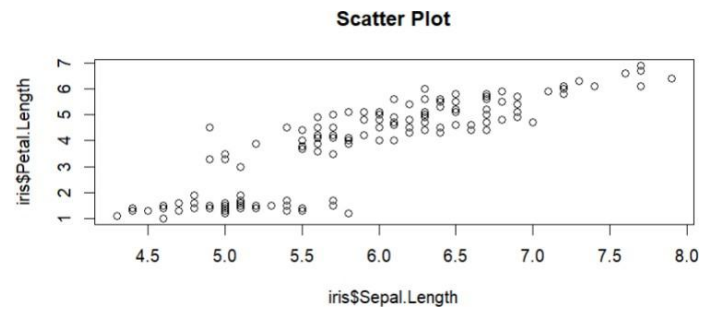
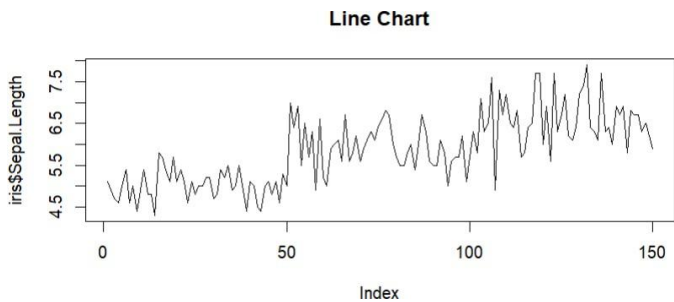
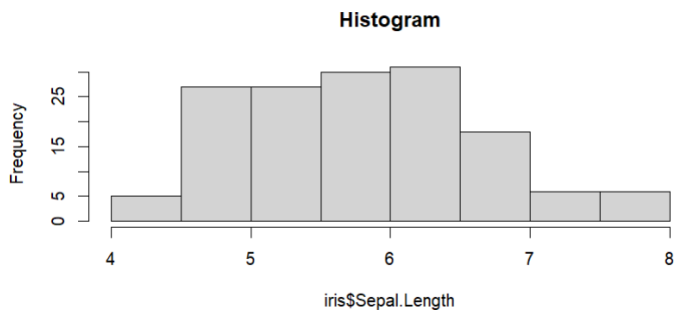
hist(iris$Sepal.Length, main="Histogram")

barplot(table(iris$Species), main="Bar Plot")

plot(iris$Sepal.Length, type="l", main="Line Chart")

plot(iris$Sepal.Length, iris$Petal.Length,
     main="Scatter Plot")
```





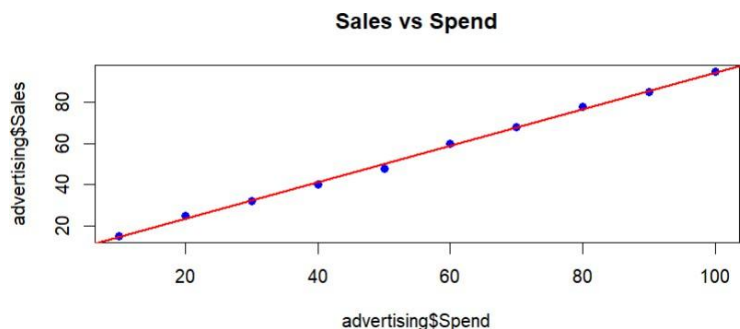
19. Build a regression model on advertising dataset (Sales ~ Spend) and predict Sales.

```
advertising <- data.frame(
  Spend = c(10,20,30,40,50,60,70,80,90,100),
  Sales = c(15,25,32,40,48,60,68,78,85,95)
)

print(advertising)
model <- lm(Sales ~ Spend, data = advertising)
print(summary(model))
new_data <- data.frame(Spend = c(25,55,75))
prediction <- predict(model, new_data)
print("Predicted Sales")

print(prediction)
```

```
Console Terminal Background Jobs
R 4.6.1 C:/Users/DEEKSHANA PRABHU/Desktop
> prediction <- predict(model, new_data)
> print("Predicted Sales")
[1] "Predicted Sales"
> print(prediction)
      1      2      3
27.94545 54.60000 72.36970
```



20. Create multiple regression model using ChickWeight dataset with “Time” and “Diet” as predictors; predict weight and compute model error.

```
data("ChickWeight")

model <- lm(
  weight ~ Time + Diet,
  data = ChickWeight
)

summary(model)

prediction <- predict(model)
print("Predicted Weight")

head(prediction)
actual <- ChickWeight$weight

mse <- mean((actual - prediction)^2)

rmse <- sqrt(mse)

cat("Mean Squared Error =", mse, "\n")

cat("Root Mean Squared Error =", rmse, "\n")
```

```
Console Terminal Background Jobs
R 4.6.1 - C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING/R PROG LAB/Exp 13/
Residuals:
    Min       1Q   Median       3Q      Max
-136.851  -17.151   -2.595   15.033  141.816

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  10.9244     3.3607   3.251  0.00122 **
Time          8.7505     0.2218  39.451 < 2e-16 ***
Diet2        16.1661     4.0858   3.957  8.56e-05 ***
Diet3        36.4994     4.0858   8.933 < 2e-16 ***
Diet4        30.2335     4.1075   7.361  6.39e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 35.99 on 573 degrees of freedom
Multiple R-squared:  0.7453,    Adjusted R-squared:  0.7435
F-statistic: 419.2 on 4 and 573 DF,  p-value: < 2.2e-16

> prediction <- predict(model)
>
> print("Predicted Weight")
[1] "Predicted Weight"
>
> head(prediction)
      1      2      3      4      5      6
10.92439 28.42537 45.92636 63.42734 80.92833 98.42931
>
> actual <- ChickWeight$weight
> mse <- mean((actual - prediction)^2)
>
> rmse <- sqrt(mse)
>
> cat("Mean Squared Error =", mse, "\n")
Mean Squared Error = 1284.319
>
> cat("Root Mean Squared Error =", rmse, "\n")
Root Mean Squared Error = 35.83739
```

21. Randomly split iris dataset into train/test (80/20), build logistic regression (Species ~ Petal.Length + Petal.Width), predict, and evaluate with confusion matrix.

```
data(iris)

set.seed(123)
n <- sample(1:nrow(iris), 0.8*nrow(iris))
train <- iris[n, ]
test <- iris[-n, ]

model <- glm(Species ~ Petal.Length + Petal.Width,
  data=train, family=binomial)

pred <- predict(model, test, type="response")
pred <- ifelse(pred > 0.5, "versicolor", "setosa")

table(Actual=test$Species, Predicted=pred)
```

```
Console Terminal Background Jobs
R 4.6.1 - C:/Users/DEEKSHANA PRABHU/Desktop/R PROGRAMMING
>
> table(Actual=test$Species, Predicted=pred)
      Predicted
Actual      setosa versicolor
setosa      10         0
versicolor   0        15
virginica    0         5
```